GEAR: CO-EVOLVING GROUNDED VLM AGENTS WITH ENVIRONMENT-ALIGNED RUBRICS

Anonymous authors
Paper under double-blind review

ABSTRACT

Training multi-turn VLM agents from environment feedback remains challenging because outcome rewards indicate whether a trajectory succeeds, but provide limited supervision for why intermediate visual reasoning steps and actions succeed or fail. Existing dense process rewards are often manually specified or fixed, which makes them difficult to adapt to the evolving trajectory distribution induced by RL. We propose **GEAR** (**G**rounded **E**nvironment-**A**ligned **R**ubrics), a co-evolutionary training framework that jointly optimizes a VLM policy and a rubricator that self-generates grounded rubrics for spatial grounding, action legality, temporal consistency, and transition progress. Each rubric is used as an auxiliary process reward only when its satisfaction is statistically aligned with grounded environment feedback, such as task success, return, legality, or progress, which prevents self-generated supervision from drifting toward textual self-consistency. Experiments on multi-turn VLM agent benchmarks show that **GEAR** improves over SFT and outcome-centric RL baselines, with ablations confirming that environment-aligned rubric evolution is important for stable and interpretable policy improvement.

1 INTRODUCTION

With the rapid advancement of Vision-Language Model (VLM) agents, they are increasingly expected to solve interactive tasks by perceiving visual states, reasoning over histories, and acting within dynamic environments Ma et al. (2026); Shinde et al. (2025). Unlike static vision-language tasks Liu et al. (2025a); Tu et al. (2023), agent learning requires sequential decisions under partial observability, where each observation provides only a limited view of the underlying state. Consequently, recent research Chen et al. (2025a); Zhu et al. (2025) emphasizes world modeling, which infers latent states and predicts transitions, as a key mechanism for reliable performance. However, environmental rewards typically provide sparse supervision focused on trajectory-level success, offering limited guidance for intermediate reasoning steps. Thus, outcome-centric RL may improve task success while leaving the grounded decision-making process underconstrained Wu et al. (2025).

Recent methods address this issue by introducing dense process rewards that supervise intermediate properties Cui et al. (2025); Ye et al. (2025a); Zhang et al. (2025), such as state estimation Chi et al. (2025), action validity Zhang et al. (2026d), or transition prediction Wang et al. (2025a). These signals can provide more informative guidance than sparse task success, but they are typically manually designed or fixed before RL begins. As a result, predefined criteria can be rigid; as the policy improves and the trajectory distribution shifts, static rewards may saturate or fail to distinguish high-quality reasoning Hübotter et al. (2026); Ye et al. (2026). Consequently, multi-turn VLM agents require process supervision that is not only grounded in environment interaction, but also adaptive to the evolving trajectory distribution induced by RL Wang et al. (2025a). Furthermore, while some recent approaches leverage proprietary expert models through APIs to provide expert-level supervision Raghavendra et al. (2026), such supervision incurs prohibitive financial costs and inference latency during the massive exploration required for RL. This limitation highlights the need for a self-sufficient, parameter-efficient paradigm in which a single model can simultaneously act in the environment and adaptively supervise its own learning.

Rubrics provide a promising interface for such supervision because they express evaluation criteria in an explicit and checkable form Gunjal et al. (2025); Pathak et al. (2025). Recent works have shown that rubrics in LLMs can provide structured reward signals beyond binary correctness, and

Reviewers: please read the Reviewer Guidelines (iclr.cc/Conferences/2027/ReviewerGuidelines) and the AI Policy for Reviewers (iclr.cc/Conferences/2027/AIPolicyForReviewers). If you used AI to expand, edit, or polish your review, please provide the input text to the LLM. Better still, consider skipping the LLM and submitting your original text: we, and the authors, are much more interested in your unedited thoughts than in what an LLM has to say. AI-assisted or not, you are putting your name and reputation behind your review: LLM-generated falsehoods, hallucinations or misrepresentations are subject to disciplinary action, which may include desk-rejecting all papers you have authored.

self-evolving rubric methods suggest that models can generate and refine their own supervision criteria for LLM reasoning Shao et al. (2025); Sharma et al. (2025); Siro et al. (2026). However, directly extending these ideas to VLM agents introduces a new grounding problem Chen et al. (2025b; 2026). A rubric may sound reasonable in language, but fail to predict whether an action is valid, safe, or useful in the environment. In interactive settings, a useful rubric should be more than a textual preference; it should serve as an environment-grounded hypothesis about which process property predicts successful interaction. This motivates our central question: ***can environment interaction turn self-generated rubrics into adaptive process rewards for VLM agents?***

To answer this question, we propose **GEAR** (**G**rounded **E**nvironment-**A**ligned **R**ubrics), a co-evolutionary training framework for multi-turn VLM agents. GEAR contains two coupled roles: a solver that generates visual reasoning traces and executable actions, and a rubricator that self-generates grounded rubrics from trajectories and environment feedback. These rubrics describe process-level properties that are central to grounded agent learning, including spatial grounding, action legality, temporal consistency, and transition progress. Unlike static process rewards, self-generated rubrics are not trusted immediately. GEAR validates each rubric by measuring whether rubric satisfaction is statistically aligned with grounded environment feedback, such as task success, return, action legality, or progress toward a goal. Only validated rubrics are used as auxiliary rewards for policy optimization, preventing self-generated supervision from drifting toward textual self-consistency.

GEAR follows a two-stage training procedure. Expert-alignment SFT first initializes the solver's ability to learn from rubric-based supervision and the rubricator's ability to generate grounded criteria. During subsequent RL, the solver learns from environment rewards augmented by validated rubric rewards, while the rubricator adapts its criteria to trajectories generated by the evolving policy. Environment-aligned validation and reward scheduling control the influence of noisy or misaligned rubrics, allowing process supervision to evolve with agent behavior. We evaluate GEAR across multi-turn VLM agent benchmarks against SFT, outcome-centric RL, and proprietary model references. With a 3B backbone, GEAR achieves an overall score of 0.87, outperforming trained baselines using the same backbone and exceeding the evaluated zero-shot proprietary model references. In 3D Navigation, it attains a success rate of 0.87, suggesting benefits for spatial reasoning and long-horizon decision-making. Ablations support the contributions of rubric validation and evolution, while qualitative analyses show that the evolved rubrics capture failure modes such as invalid actions, stale beliefs, and insufficient transition progress. Together, these results demonstrate the potential of environment-validated, self-generated rubrics as adaptive process supervision for VLM agents.

2 Related Works

Self-Evolving Agentic Learning have emerged as a pivotal paradigm for achieving general artificial intelligence by minimizing human intervention through closed-loop optimization Gao et al. (2025); Xiang et al. (2026). Early research treated prompts as discrete variables, optimizing them via task-level scoring or iterative instruction proposals based on prior performance. While effective, these methods often overfit to fixed protocols and lack online adaptivity. Recent advancements leverage self-play Lu et al. (2025); Xia et al. (2025); Zhang et al. (2026c;b) and self-training Lin et al. (2025); Wang et al. (2026); Zhai et al. (2025), where a single policy model generates its own training data or provides reward signals during learning. To escape local optima, population-based algorithms employ LLM-based mutations and fitness selection to evolve prompts and agentic workflows Liu et al. (2025b). Despite these advances, current methods often suffer from rigidity due to fixed heuristic loops or a heavy reliance on ground-truth signals for failure diagnosis Xu et al. (2026). Unlike previous work that may drift toward textual self-consistency, GEAR ensures process rewards are statistically aligned with grounded feedback environment-aligned self-generated rubrics, achieving spatial mastery that surpasses leading proprietary models.

Rubrics-based Reinforcement Learning. Structured rubrics have emerged as a pivotal paradigm for providing fine-grained, interpretable reward signals in reinforcement learning Pathak et al. (2025); Huang et al. (2025). Most existing research Li et al. (2026a;b); Raghavendra et al. (2026); Zhang et al. (2026a) established the utility of LLM-based judges to decompose high-level objectives into checkable, multi-facet criteria. However, these methods heavily rely on strong proprietary models as graders or expert-tailored data, which introduces prohibitive costs and limits overall scalability Wang et al. (2025b). Recent trends toward dynamic rubric evolution Sheng et al. (2026); Xu et al.

(2026); Ye et al. (2025b) address shifts in model behavior; these approaches often incur significant computational overhead and remain largely confined to textual logic Gupta et al. (2025). In contrast, GEAR introduces grounded, environment-aligned rubrics that co-evolve with the agent's policy. By validating self-generated rubrics against environment feedback, GEAR provides adaptive process supervision throughout online RL with a shared VLM backbone.

3 PRELIMINARIES

We first formulate multi-turn VLM agent learning and then clarify why outcome rewards alone provide limited guidance for grounded reasoning. We use these concepts to motivate process-level supervision, which is the basis of our method.

3.1 MULTI-TURN VLM AGENT LEARNING

We consider interactive vision-language agent tasks in which an agent solves a task through multiple rounds of visual observation, language reasoning, and executable action. Such tasks can be formulated as a POMDP,

$$\mathcal{M} = (\mathcal{S}, \mathcal{O}, \mathcal{A}, P, \Omega, R, \gamma), \tag{1}$$

where $\mathcal{S}, \mathcal{O}, \mathcal{A}$ denote the state, observation, and action spaces, P, Ω, R denote transition, observation, and reward functions, and $\gamma \in [0, 1]$ is the discount factor.

At turn t, the agent receives a visual observation $o_t \in \mathcal{O}$, which provides only partial information about the latent state s_t. Given a task instruction x and the interaction history

$$h_t = (x, o_0, a_0, r_0, \ldots, o_{t-1}, a_{t-1}, r_{t-1}, o_t), \tag{2}$$

a VLM policy π_θ generates a reasoning trace c_t and an executable action a_t:

$$c_t, a_t \sim \pi_\theta(\cdot \mid h_t). \tag{3}$$

The action is executed in the environment, which produces a reward r_t and a new observation o_{t+1}. Repeating this process for T turns yields a trajectory

$$\tau = (x, o_0, c_0, a_0, r_0, \ldots, o_{T-1}, c_{T-1}, a_{T-1}, r_{T-1}, o_T). \tag{4}$$

The standard RL objective is to optimize the policy to maximize the expected discounted return:

$$\max_\theta \ \mathbb{E}_{\tau \sim \pi_\theta, P, \Omega} \left[\sum_{t=0}^{T-1} \gamma^t r_t \right]. \tag{5}$$

Unlike static vision-language question answering, this setting requires the agent to act under partial observability and reason about visual states, action constraints, and future transitions.

3.2 OUTCOME REWARDS AND PROCESS SUPERVISION

A common way to train VLM agents is to optimize environment-derived rewards. In sparse-reward tasks, the reward may only indicate whether the final trajectory succeeds:

$$r^{\text{env}}(\tau) = \mathbb{I}[\tau \text{ succeeds}], \tag{6}$$

whereas denser environments may additionally provide step-level feedback, such as action legality, collision penalties, or progress toward a goal:

$$r_t^{\text{env}} = R(s_t, a_t, s_{t+1}). \tag{7}$$

Such rewards are grounded because they are determined by environment dynamics rather than by the language model itself.

However, outcome rewards provide limited guidance on how an agent should interpret visual states, update its reasoning, or choose actions across intermediate steps. This creates a credit-assignment problem in multi-turn VLM agent learning. In this work, we use rubrics as explicit and checkable process criteria, while requiring them to be validated by environment feedback rather than treated as static textual preferences.

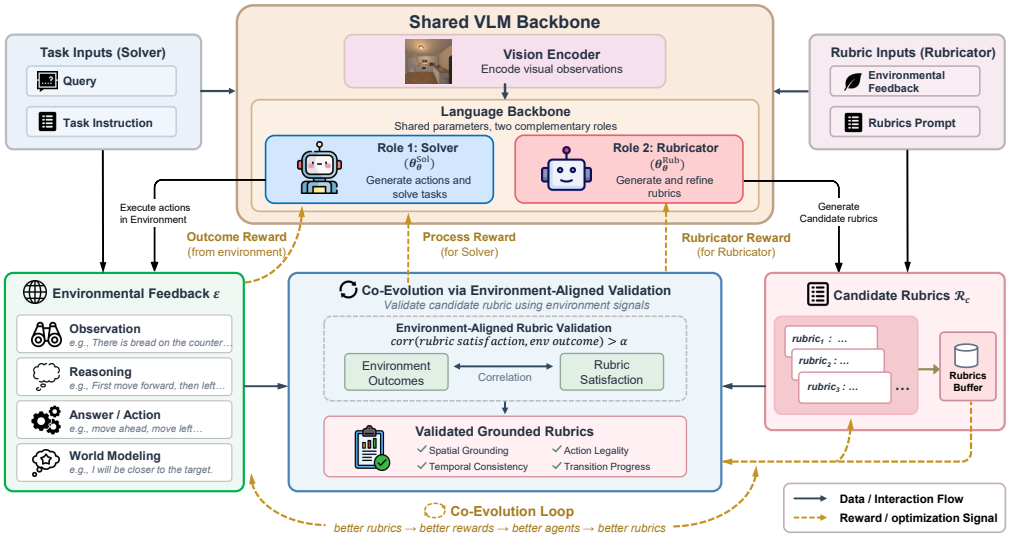


Figure 1: The overview of the **G**rounded **E**nvironment-**A**ligned **R**ubrics (**GEAR**) framework. A shared VLM backbone instantiates two roles: a Solverthat generates reasoning traces and actions, and a Rubricator that proposes candidate criteria. Through a co-evolution loop, these rubrics are statistically validated against Environmental Feedback to ensure grounding. Validated rubrics—addressing spatial, legal, temporal, and progress dimensions —provide dense Criteria Rewards (r_t^{crit}) alongside sparse Outcome Rewards (r_t^{env}) to supervise policy optimization.

4 METHOD

We introduce **GEAR** (**G**rounded **E**nvironment-**A**ligned **R**ubrics), a co-evolutionary framework that converts environment feedback into validated process supervision for multi-turn VLM agents. GEAR uses a shared VLM under two role-specific prompts: a *solver* that generates reasoning traces and executable actions, and a *rubricator* that proposes candidate process rubrics from interaction trajectories and environment feedback.

4.1 GEAR OVERVIEW

Formally, we instantiate a shared policy π_θ under two roles, denoted by $\pi_\theta^{\mathrm{Sol}}$ and $\pi_\theta^{\mathrm{Rub}}$. Both roles share the same parameter set θ and are distinguished only by their role-specific prompts.

Given a task instruction x and an interaction history h_t, the solver generates a reasoning trace and an executable action:

$$c_t, a_t \sim \pi_\theta^{\mathrm{Sol}}(\cdot \mid x, h_t). \tag{8}$$

The action is executed in the environment, producing a new observation o_{t+1} and an environment reward r_t^{env}. After collecting trajectories, the rubricator proposes a set of candidate rubrics:

$$\widehat{\mathcal{R}} \sim \pi_\theta^{\mathrm{Rub}}(\cdot \mid x, \tau, z(\tau)), \qquad \widehat{\mathcal{R}} = \{\hat{\rho}_k\}_{k=1}^{K}, \tag{9}$$

where τ denotes an interaction trajectory and $z(\tau)$ denotes grounded feedback derived from the environment, such as task success, return, action legality, or progress toward the goal.

The generated rubrics are treated only as candidates rather than directly used as rewards. GEAR retains a rubric only when its satisfaction is statistically aligned with environment feedback. Validated rubrics provide process rewards to the solver, while their validation outcomes reward the rubricator. As solver updates shift the trajectory distribution, the rubricator continually adapts its criteria to remain informative under the evolving policy, forming a co-evolutionary training loop. Figure 1 summarizes this process.

4.2 GROUNDED RUBRIC GENERATION AND COLD START

We parameterize each candidate rubric as

$$\rho_k = (d_k, q_k, w_k), \tag{10}$$

where d_k denotes the rubric dimension, q_k is a natural-language or structured criterion, and $w_k \in \mathbb{R}$ is its associated weight.

We consider four dimensions of grounded interaction: `SpatialGrounding`, `ActionLegality`, `TemporalConsistency`, and `TransitionProgress`. `SpatialGrounding` evaluates whether the reasoning correctly reflects the visual layout, object relations, and obstacle positions. `ActionLegality` evaluates whether the executable action obeys environment constraints and avoids invalid transitions, such as collisions or falling into traps. `TemporalConsistency` evaluates whether the reasoning is updated according to the interaction history rather than repeating stale beliefs. `TransitionProgress` evaluates whether the action is expected to move the agent toward a more favorable future state, such as reducing the distance to the goal or increasing task progress. Together, these dimensions constrain rubric generation toward grounded interaction properties rather than generic textual qualities.

Before co-evolutionary RL, we initialize both roles with expert-alignment SFT. Details of the cold-start procedure are provided in Appendix B.

4.3 ENVIRONMENT-ALIGNED RUBRIC VALIDATION

Self-generated rubrics may be vague, saturated, or disconnected from actual environment outcomes. GEAR therefore validates each candidate rubric by testing whether its satisfaction is statistically aligned with grounded environment feedback.

Given a trajectory τ_i and a rubric ρ_k, a verifier $\mathcal{V}_\phi$ evaluates whether the trajectory satisfies the rubric:

$$v_{i,k} = \mathcal{V}_\phi(\rho_k, \tau_i) \in 0, 1. \tag{11}$$

We use a frozen, offline-distilled verifier throughout RL, which does not receive the environment outcome target z during online verification; see Appendix B.2 for distillation and evaluation details.

When step-level evaluation is available, rubric satisfaction can also be evaluated on a local transition:

$$v_{t,k} = \mathcal{V}_\phi(\rho_k, h_t, c_t, a_t, o_{t+1}). \tag{12}$$

Here, $v_{i,k}$ is used to validate rubrics across trajectories, whereas $v_{t,k}$ is used to construct the turn-level criterion reward below; trajectory-level satisfaction is not directly broadcast as per-step reward. See Appendix A for implementation details.

Let $z_i = g_{\mathcal{E}}(\tau_i)$ denote an environment-derived target for trajectory τ_i, such as final success, return, legality, or progress. For each rubric ρ_k, we define its satisfaction vector and the corresponding feedback vector as

$$\mathbf{v}_k = [v_{1,k}, \ldots, v_{N,k}], \qquad \mathbf{z} = [z_1, \ldots, z_N]. \tag{13}$$

The same criterion ρ_k is held fixed across the N trajectories used to compute its correlation. For each rollout comparison group, a rubric is retained if it is both environment-aligned and non-degenerate:

$$\rho_k \in \mathcal{R}^{\text{valid}} \iff \text{corr}(\mathbf{v}_k, \mathbf{z}) > \alpha \ \wedge \ \text{std}(\mathbf{v}_k) > 0 \ \wedge \ \text{std}(\mathbf{z}) > 0, \tag{14}$$

where $\text{corr}(\cdot, \cdot)$ denotes the Pearson correlation coefficient and α is the correlation threshold. The first condition selects rubrics whose satisfaction is positively associated with environment outcomes, while the remaining conditions exclude constant rubric satisfaction scores and environment outcomes.

Estimating correlation from a single batch can be unstable when outcome variance is low. GEAR separately maintains a lightweight correlation tracker for each rubric dimension. For dimension d, the tracker stores recent satisfaction-feedback pairs:

$$\mathcal{B}_d \leftarrow \mathcal{B}_d \cup (v_{i,k}, z_i) : d_k = d, \tag{15}$$

and computes a smoothed alignment estimate:

$$\bar{c}_d \leftarrow \text{EMA}\big(\text{corr}(\mathcal{B}_d)\big). \tag{16}$$

Rubric validity is determined at the individual-rubric level, while the dimension-level EMA is used only for monitoring and reward scheduling; historical rule-level fallback for degenerate rollout groups is detailed in Appendix A.

4.4 Co-Evolutionary Rewards and Optimization

GEAR assigns role-specific rewards to the solver and the rubricator. The solver is optimized with environment rewards augmented by validated process supervision, whereas the rubricator is rewarded for generating rubrics that pass environment-aligned validation. This separates rewards for acting in the environment from rewards for generating supervision.

For the solver, validated rubrics are aggregated into a criterion reward at each interaction turn:

$$r_t^{\text{crit}} = \text{norm}\left(\sum_{\rho_k \in \mathcal{R}_{\text{valid}}} w_k \cdot \mathcal{V}_\phi(\rho_k, h_t, c_t, a_t, o_{t+1})\right). \tag{17}$$

The total solver reward at training update u is

$$r_t^{\text{sol}} = r_t^{\text{env}} + \lambda^{(u)} r_t^{\text{crit}}, \tag{18}$$

where $\lambda^{(u)}$ controls the strength of rubric-based process supervision. The criterion reward does not replace environment feedback; rather, it densifies sparse outcome supervision with validated signals indicating which reasoning and action properties are associated with successful interaction.

To prevent noisy rubrics from destabilizing early policy learning, GEAR gradually introduces criterion supervision during warmup using

$$\lambda^{(u)} = \min\left(1, \frac{u}{U_{\text{warm}}}\right)\lambda_{\max}, \tag{19}$$

where u denotes the training update and U_{warm} is the warmup duration.

After warmup, GEAR applies correlation gating using the smoothed environment-alignment estimates. If the moving correlation estimate remains below the threshold for several updates, the criterion reward weight is decayed:

$$\lambda^{(u)} \leftarrow \kappa\lambda^{(u)}, \qquad 0 < \kappa < 1. \tag{20}$$

where κ denotes the decay factor. When the correlation remains persistently low, we set $\lambda^{(u)} = 0$ and fall back to pure environment reward. This safeguard reduces the risk that the solver and rubricator reinforce internally consistent but environmentally ungrounded behaviors.

For the rubricator, the alignment reward is defined by the fraction of generated rubrics that pass environment-aligned validation:

$$r_{\text{align}}^{\text{rub}} = \frac{K_{\text{valid}}}{K}, \tag{21}$$

where K is the number of rubric slots specified by the task schema, one for each core dimension, and K_{valid} counts slots containing a valid rubric. Missing or malformed entries receive zero alignment credit. Task-specific rubric schemas are provided in Appendix G. To ensure stable parsing, we additionally assign a format reward:

$$r_{\text{format}}^{\text{rub}} = \mathbb{I}[\text{parse}(\hat{\mathcal{R}})]. \tag{22}$$

The total rubricator reward is

$$r^{\text{rub}} = r_{\text{align}}^{\text{rub}} + \eta_{\text{format}} r_{\text{format}}^{\text{rub}}, \tag{23}$$

where η_{format} controls the weight of format correctness.

We optimize both roles with standard PPO. Solver advantages are computed from r_t^{sol}, while rubricator advantages are computed from r^{rub}. Because the solver and rubricator share the same parameter set θ, gradients from both role-specific objectives are jointly accumulated to update the shared VLM policy:

$$\mathcal{J}(\theta) = \mathcal{J}_{\text{sol}}(\theta) + \omega\mathcal{J}_{\text{rub}}(\theta), \tag{24}$$

where $\mathcal{J}_{\text{sol}}(\theta)$ and $\mathcal{J}_{\text{rub}}(\theta)$ denote the standard PPO objectives for the solver and rubricator roles, respectively, and ω controls the relative contribution of rubricator updates. We do not assume the two objectives are gradient-aligned; Appendix C.3 compares shared and separate optimization as an efficiency–performance trade-off.

Table 1: Performance comparison across diverse agentic benchmarks. We report the average Success Rate for cognitive puzzles (Sokoban, FrozenLake) and embodied control tasks (Navigation, PrimitiveSkill), alongside perceptual similarity (mean of DINO and DreamSim scores) for the SVG reconstruction task. All trained models utilize Qwen2.5-VL-3B as the common backbone. **Bold** denotes the optimal performance among trained models. **Overall** averages the five task-level scores.

Model/Method	Sokoban	FrozenLake	Navigation			PrimitiveSkill					SVG			Overall
			Base	Common	**Average**	Place	Stack	Drawer	Align	**Average**	Dino	DreamSim	**Average**	
Open-Source Models														
Qwen2.5-VL-72B (Bai et al., 2023)	0.18	0.44	0.72	0.75	0.73	**1.00**	0.50	0.00	**1.00**	0.44	0.89	0.62	0.76	0.51
Qwen2.5-VL-7B (Bai et al., 2023)	0.13	0.14	0.28	0.39	0.34	0.00	0.00	0.00	0.75	0.19	0.83	0.28	0.55	0.27
Qwen2.5-VL-3B (Bai et al., 2023)	0.14	0.14	0.22	0.27	0.24	0.00	0.00	0.00	0.00	0.00	0.80	0.27	0.54	0.21
VLM-R1-3B (Shen et al., 2025)	0.13	0.13	0.31	0.34	0.33	0.00	0.00	0.00	0.00	0.00	0.81	0.28	0.55	0.23
Proprietary Models														
Gemini 2.5 Pro(Google, 2025)	0.58	0.78	0.63	0.63	0.63	0.63	0.63	0.00	0.75	0.50	0.93	0.78	0.86	0.67
Claude 4.5 Sonnet (Anthropic, 2025)	0.31	0.80	0.67	0.67	0.67	0.63	0.50	0.00	**1.00**	0.53	**0.95**	**0.81**	**0.88**	0.64
GPT-5 (Singh et al., 2025)	**0.70**	0.77	0.75	0.81	0.78	**1.00**	0.63	0.00	**1.00**	0.66	**0.95**	0.75	0.85	0.75
RL & World Modeling Baselines (Backbone: Qwen2.5-VL-3B)														
Vanilla-PPO Wang et al. (2025a)	0.18	0.21	0.32	0.25	0.29	0.00	0.00	0.00	0.00	0.00	0.83	0.44	0.64	0.26
VAGEN-Full Wang et al. (2025a)	0.79	0.74	0.80	0.81	0.81	**1.00**	**0.88**	**1.00**	**1.00**	0.97	0.91	0.67	0.79	0.82
Rubrics-based World Modeling Variants (Backbone: Qwen2.5-VL-3B)														
SFT	0.30	0.21	0.43	0.39	0.41	0.00	0.00	0.00	0.50	0.13	0.85	0.48	0.67	0.35
Training-Free [1]	0.42	0.26	0.45	0.42	0.44	0.00	0.00	0.00	0.13	0.04	0.86	0.54	0.70	0.37
GEAR (Ours)	**0.86**	**0.82**	**0.85**	**0.88**	**0.87**	**1.00**	**0.88**	**1.00**	**1.00**	**0.97**	0.93	0.72	0.83	**0.87**

5 EXPERIMENTS

5.1 EXPERIMENTAL SETUP

Environments. We evaluate GEAR across a diverse suite of five distinct tasks, spanning 2D cognitive grid puzzles (Sokoban and FrozenLake), embodied 3D control (Navigation and PrimitiveSkill), SVG reconstruction. These environments assess the agent's ability to perform multi-step planning, spatial grounding, and cross-modal alignment. Detailed task descriptions and environment settings are provided in Appendix F.

Metrics. We assess agent performance in puzzles and manipulation via Success Rate (SR), using a sparse reward structure triggered solely by goal achievement. For SVG generation, we calculate a multi-faceted similarity score by combining DINO Caron et al. (2021) and DreamSim Fu et al. (2023) to evaluate visual fidelity. Refer to Appendix E for the detailed metric configuration.

Baseline. To comprehensively evaluate GEAR, we compare it against three categories of baselines. First, we benchmark against foundation models, including open-source Qwen2.5-VL (3B, 7B, 72B) Bai et al. (2023). We also include zero-shot evaluations of proprietary models as reference results like GPT-5 Singh et al. (2025) and Gemini 2.5 Pro Google (2025). Second, we include RL and world-modeling frameworks, specifically Vanilla-PPO and VAGEN-Full Wang et al. (2025a). Finally, we evaluate process-level supervision variants, including SFT and a Training-Free version using static rubrics. To ensure a fair comparison, all RL-based methods and variants utilize the Qwen2.5-VL-3B backbone.

Implementation Details. We train GEAR using decoupled learning rates to optimize different architectural components: 1×10^{-6} for the prediction loss and RL actor, while the critic uses a higher rate of 1×10^{-5}. All experiments are conducted with a global batch size of 128. To mitigate the non-stationarity and high variance typical of RL, we normalize intrinsic rewards by the running standard deviation of their returns.We fix the rubricator optimization factor at $\omega = 0.01$ and the correlation threshold at $\alpha = 0.1$ across all tasks. During rollouts and inference, we employ a VLM generation temperature of 0.7 to balance diversity and precision. For the verifier, we distill Qwen2.5-VL-3B based on the verification responses from Gemini 2.5 Pro. Verifier training and evaluation are provided in Appendix B. For the complete hyper-parameter configurations, please refer to Appendix F and G.

[1] In the training-free paradigm, we utilize Gemini 2.5 Pro as a rubricator, where the backbone model explores eight candidate trajectory directions per iteration. The optimal trajectory is subsequently selected based on the highest score yielded by the generated rubrics.

Table 2: Performance comparison of different components in GEAR.

Model/Method	Sokoban	FrozenLake	Navigation			SVG		
			Base	Common	**Average**	Dino	DreamSim	**Average**
Baseline	0.18	0.21	0.32	0.25	0.29	0.83	0.44	0.64
w/o SFT Cold-Start	0.62	0.66	0.68	0.64	0.66	0.80	0.46	0.63
w/o Rubric Validation	0.79	0.76	0.76	0.80	0.78	0.90	0.68	0.79
w/o Rubric Evolution	0.78	0.78	0.75	0.77	0.76	0.92	0.69	0.81
w/o Rubric Scheduling	0.82	0.80	0.82	0.84	0.84	0.93	0.71	0.82
GEAR (Ours)	**0.86**	**0.82**	**0.85**	**0.88**	**0.87**	**0.93**	**0.72**	**0.83**

5.2 Main Results

As shown in Table 1, GEAR achieves the highest overall score of 0.87 on the evaluated benchmarks with a compact 3B backbone, surpassing the zero-shot results of GPT-5 (0.75) and Claude Sonnet 4.5 (0.64). This breakthrough is driven by a co-evolutionary paradigm that internalizes environment-aligned rubrics as dense process-level supervision, effectively resolving the multi-turn credit-assignment problem. Specifically, GEAR demonstrates remarkable versatility across diverse agentic tasks. ***Cognitive Logic Mastery:*** GEAR attains an 0.86 success rate in Sokoban and 0.82 in FrozenLake, outmaneuvering massive proprietary models in intricate planning. These results substantiate that parameter-efficient agents can overcome complex reasoning bottlenecks when intermediate trajectories are guided by grounded, self-generated rubrics. ***Physical & Spatial Grounding:*** With a dominant 0.97 success rate in PrimitiveSkill and 0.87 in Navigation, GEAR proves that anchoring reasoning in physical constraints allows compact models to master complex manipulation and long-horizon spatial memory more effectively than frontier generalized engines. ***Cross-Modal Precision:*** In SVG reconstruction, GEAR achieves a 0.83 similarity score, showcasing high-precision alignment between abstract code and visual targets. This demonstrates that our process-level supervision effectively bridges the gap between high-level reasoning and generative execution. Overall, GEAR's core contribution is its environment-correlated verification protocol, which prevents "self-consistency drift" by anchoring reasoning in empirical feedback. This innovation allows a 3B-parameter model to master long-horizon dynamics previously considered the exclusive domain of large-scale proprietary systems. By bridging the gap between reasoning and execution, GEAR represents a significant paradigm shift in training autonomous multimodal agents. Moreover, three-seed results for the main trained baselines and GEAR are provided in Appendix C.1.

5.3 Ablation Studies

Effectiveness of Components. To systematically evaluate the contribution of each design element, we compare GEAR against several truncated variants. The results in Table 2 removing each component reduces performance on the evaluated tasks. Specifically, ***Baseline (Vanilla-PPO)*** yields the lowest performance by relying solely on sparse outcome rewards, highlighting the severe credit-assignment challenges in multi-turn VLM tasks. Performance drops sharply without ***SFT Cold-Start***, proving that a semantic foundation is required for the model to internalize rubric logic. Removing ***Rubric Validation*** leads to a significant decline, as the agent fails to distinguish between linguistically plausible reasoning and actions that drive actual environment success. Furthermore, lacking ***Rubric Evolution***, thus utilizing static criteria, prevents the model from adapting to shifting trajectory distributions during RL, which stifles long-term policy improvement. Finally, the absence of warmup and gating mechanisms in ***Rubric Scheduling*** allows early exploration noise to destabilize training. By integrating all components, our framework achieves optimal performance across all benchmarks, demonstrating that validated, evolving rubrics provide a superior paradigm for grounded process-level supervision.

5.3.1 Effectiveness of Grounded rubrics

Effectiveness of Grounded rubrics. We evaluate GEAR across four dimensions: SpatialGrounding, ActionLegality, TemporalConsistency, and TransitionProgress. As illustrated in Figure 3, while individual rubrics significantly boost performance—notably raising Sokoban SR to 0.83—their integration in the full framework yields the greatest gains. These dimensions are highly complementary:

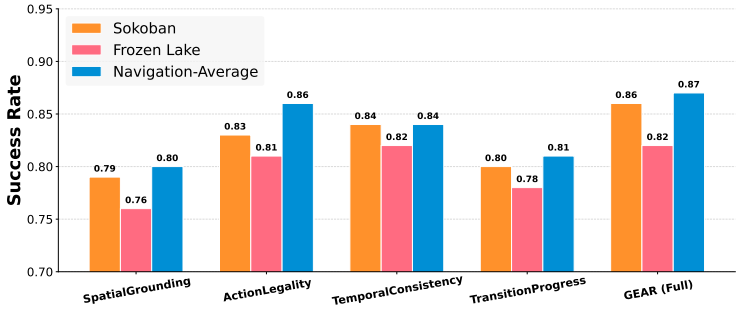
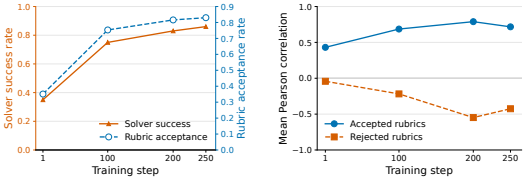


Figure 3: Impact of grounded rubric dimensions across various agentic tasks.

ActionLegality and SpatialGrounding enforce vital visual-action constraints, TemporalConsistency (0.84 in Navigation) evaluates updates to prior reasoning, and TransitionProgress evaluates progress toward the goal. Ultimately, GEAR (Full) achieves a peak SR of 0.87,showing that these complementary reward signals help a compact 3B model address multi-turn credit assignment and surpass the zero-shot proprietary reference scores.

Rubric Validation Dynamics. We track rubric validation in Figure 2 during optimization. The rubric acceptance rate increases from 35.1% to 83.0%, accompanied by an improvement in solver success from 0.35 to 0.86. The initial acceptance rate indicates that validation admits candidate supervision despite sparse outcome feedback. Across the measured training steps, accepted rubrics maintain positive mean alignment with environment outcomes, whereas rejected rubrics exhibit negative mean alignment. This analysis illustrates GEAR's selective supervision mechanism, which continually reassesses the outcome alignment of self-generated criteria as the policy evolves.



(a) Rubric Acceptance (b) Rubric Alignment

Figure 2: Rubric Validation During Training

Computational Efficiency Analysis. As shown in Table 3, GEAR achieves a superior balance between performance and resource consumption. While Separate Optimization (Sep. Opt.) yields slightly higher success rates, it demands nearly double the memory (783.2 GB) compared to GEAR. By utilizing a Shared VLM Backbone for both solver and rubricator roles, GEAR maintains high performance (0.86 in Sokoban) with a manageable 431.5 GB footprint—a 45% reduction in peak GPU memory over Sep. Opt. This design introduces only a marginal 72 GB increase over the vanilla PPO baseline. These results prove that sophisticated, self-evolving rewards do not require prohibitive hardware. By internalizing the rubricator within a compact 3B backbone, GEAR enables high-performance agentic reasoning on standard research-grade infrastructure, significantly enhancing the scalability and accessibility of autonomous VLM training. We provide more evaluation results and hyperparameter analysis in Appendix C and D.

Method	Sokoban	FrozenLake	Mem.(GB)
Baseline	0.18	0.21	359.6
Sep. Opt.	0.87	0.84	783.2
GEAR (Ours)	**0.86**	**0.82**	**431.5**

Table 3: Trade-off Analysis of performance and computation cost.

6 Conclusion

We present GEAR (Grounded Environment-Aligned Rubrics), a co-evolutionary framework designed to circumvent the limitations of sparse rewards and rigid supervision in VLM agents learning. Its primary innovation is an environment-aligned validation mechanism that statistically anchors self-generated rubrics—covering spatial, legal, temporal, and progress dimensions—to empirical feedback. It effectively prevents "textual self-consistency drift" by ensuring that internal reasoning remains grounded in environmental dynamics. Experimental results demonstrate that GEAR establishes a new performance ceiling with a 0.87 overall score, surpassing frontier models like GPT-5 and Claude 4.5 Sonnet, demonstrating that grounded, co-evolutionary rubric supervision enables a compact agent to exceed the zero-shot scores of frontier proprietary models on the evaluated benchmarks.

REFERENCES

Anthropic. Introducing claude sonnet 4.5, 2025.

Jinze Bai, Shuai Bai, Yunfei Chu, Zeyu Cui, Kai Dang, Xiaodong Deng, Yang Fan, Wenbin Ge, Yu Han, Fei Huang, et al. Qwen technical report. *arXiv preprint arXiv:2309.16609*, 2023.

Mathilde Caron, Hugo Touvron, Ishan Misra, Hervé Jégou, Julien Mairal, Piotr Bojanowski, and Armand Joulin. Emerging properties in self-supervised vision transformers. In *Proceedings of the IEEE/CVF international conference on computer vision*, pp. 9650–9660, 2021.

Delong Chen, Theo Moutakanni, Willy Chung, Yejin Bang, Ziwei Ji, Allen Bolourchi, and Pascale Fung. Planning with reasoning using vision language world model. *arXiv preprint arXiv:2509.02722*, 2025a.

Shiqi Chen, Tongyao Zhu, Zian Wang, Jinghan Zhang, Kangrui Wang, Siyang Gao, Teng Xiao, Yee Whye Teh, Junxian He, and Manling Li. Internalizing world models via self-play finetuning for agentic rl. *arXiv preprint arXiv:2510.15047*, 2025b.

Yukun Chen, Jiaming Li, Longze Chen, Ze Gong, Jingpeng Li, Zhen Qin, Hengyu Chang, Ancheng Xu, Zhihao Yang, Hamid Alinejad-Rokny, et al. Rucl: Stratified rubric-based curriculum learning for multimodal large language model reasoning. *arXiv preprint arXiv:2602.21628*, 2026.

Haotian Chi, Zhaogeng Liu, Xing Chen, Bohao Qu, Jifeng Hu, Yuan Jiang, Hechang Chen, and Yi Chang. State transition difference prediction for deep reinforcement learning. *Pattern Recognition*, pp. 112824, 2025.

Ganqu Cui, Lifan Yuan, Zefan Wang, Hanbin Wang, Yuchen Zhang, Jiacheng Chen, Wendi Li, Bingxiang He, Yuchen Fan, Tianyu Yu, et al. Process reinforcement through implicit rewards. *arXiv preprint arXiv:2502.01456*, 2025.

Stephanie Fu, Netanel Tamir, Shobhita Sundaram, Lucy Chai, Richard Zhang, Tali Dekel, and Phillip Isola. Dreamsim: Learning new dimensions of human visual similarity using synthetic data. *arXiv preprint arXiv:2306.09344*, 2023.

Huan-ang Gao, Jiayi Geng, Wenyue Hua, Mengkang Hu, Xinzhe Juan, Hongzhang Liu, Shilong Liu, Jiahao Qiu, Xuan Qi, Yiran Wu, et al. A survey of self-evolving agents: What, when, how, and where to evolve on the path to artificial super intelligence. *arXiv preprint arXiv:2507.21046*, 2025.

Google. Gemini 2.5: Our most intelligent ai model, 2025.

Anisha Gunjal, Anthony Wang, Elaine Lau, Vaskar Nath, Yunzhong He, Bing Liu, and Sean Hendryx. Rubrics as rewards: Reinforcement learning beyond verifiable domains. *arXiv preprint arXiv:2507.17746*, 2025.

Arnav Gupta, Gurekas Singh Sahney, Hardik Rathi, Abhishek Chandwani, Ishaan Gupta, Pratik Narang, and Dhruv Kumar. Understanding virality: A rubric based vision-language model framework for short-form edutainment evaluation. *arXiv preprint arXiv:2512.21402*, 2025.

Zenan Huang, Yihong Zhuang, Guoshan Lu, Zeyu Qin, Haokai Xu, Tianyu Zhao, Ru Peng, Jiaqi Hu, Zhanming Shen, Xiaomeng Hu, et al. Reinforcement learning with rubric anchors. *arXiv preprint arXiv:2508.12790*, 2025.

Jonas Hübotter, Frederike Lübeck, Lejs Behric, Anton Baumann, Marco Bagatella, Daniel Marta, Ido Hakimi, Idan Shenfeld, Thomas Kleine Buening, Carlos Guestrin, et al. Reinforcement learning via self-distillation. *arXiv preprint arXiv:2601.20802*, 2026.

Eric Kolve, Roozbeh Mottaghi, Winson Han, Eli VanderBilt, Luca Weihs, Alvaro Herrasti, Matt Deitke, Kiana Ehsani, Daniel Gordon, Yuke Zhu, et al. Ai2-thor: An interactive 3d environment for visual ai. *arXiv preprint arXiv:1712.05474*, 2017.

Ruizhe Li, Mingxuan Du, Benfeng Xu, Chiwei Zhu, Xiaorui Wang, and Zhendong Mao. Deepresearch bench ii: Diagnosing deep research agents via rubrics from expert report. *arXiv preprint arXiv:2601.08536*, 2026a.

Sunzhu Li, Jiale Zhao, Miteto Wei, Huimin Ren, Yang Zhou, Jingwen Yang, Shunyu Liu, Kaike Zhang, and Wei Chen. Rubrichub: A comprehensive and highly discriminative rubric dataset via automated coarse-to-fine generation. *arXiv preprint arXiv:2601.08430*, 2026b.

Jiaye Lin, Yifu Guo, Yuzhen Han, Sen Hu, Ziyi Ni, Licheng Wang, Mingguang Chen, Hongzhang Liu, Ronghao Chen, Yangfan He, et al. Se-agent: Self-evolution trajectory optimization in multi-step reasoning with llm-based agents. *arXiv preprint arXiv:2508.02085*, 2025.

Feiran Liu, Yuzhe Zhang, Xinyi Huang, Yinan Peng, Xinfeng Li, Lixu Wang, Yutong Shen, Ranjie Duan, Simeng Qin, Xiaojun Jia, et al. The eye of sherlock holmes: Uncovering user private attribute profiling via vision-language model agentic framework. In *Proceedings of the 33rd ACM International Conference on Multimedia*, pp. 4875–4883, 2025a.

Huanyu Liu, Jia Li, Chang Yu, Taozhi Chen, Yihong Dong, Lecheng Wang, XiaoLong Hu, and Ge Li. Evocot: Overcoming the exploration bottleneck in reinforcement learning. *arXiv preprint arXiv:2508.07809*, 2025b.

Hongliang Lu, Yuhang Wen, Pengyu Cheng, Ruijin Ding, Jiaqi Guo, Haotian Xu, Chutian Wang, Haonan Chen, Xiaoxi Jiang, and Guanjun Jiang. Search self-play: Pushing the frontier of agent capability without supervision. *arXiv preprint arXiv:2510.18821*, 2025.

Yueen Ma, Zixing Song, Yuzheng Zhuang, Jianye Hao, and Irwin King. A survey on vision–language–action models for embodied ai. *IEEE Transactions on Neural Networks and Learning Systems*, 2026.

Soroush Nasiriany, Huihan Liu, and Yuke Zhu. Augmenting reinforcement learning with behavior primitives for diverse manipulation tasks. In *2022 International Conference on Robotics and Automation (ICRA)*, pp. 7477–7484. IEEE, 2022.

Aditya Pathak, Rachit Gandhi, Vaibhav Uttam, Arnav Ramamoorthy, Pratyush Ghosh, Aaryan Raj Jindal, Shreyash Verma, Aditya Mittal, Aashna Ased, Chirag Khatri, et al. Rubric is all you need: Improving llm-based code evaluation with question-specific rubrics. In *Proceedings of the 2025 ACM Conference on International Computing Education Research V. 1*, pp. 181–195, 2025.

Mohit Raghavendra, Anisha Gunjal, Bing Liu, and Yunzhong He. Agentic rubrics as contextual verifiers for swe agents. *arXiv preprint arXiv:2601.04171*, 2026.

Juan A Rodriguez, Abhay Puri, Shubham Agarwal, Issam H Laradji, Pau Rodriguez, Sai Rajeswar, David Vazquez, Christopher Pal, and Marco Pedersoli. Starvector: Generating scalable vector graphics code from images and text. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pp. 16175–16186, 2025.

M.-P. B Schrader. gym-sokoban, 2018. URL `https://github.com/mpSchrader/gym-sokoban`.

Rulin Shao, Akari Asai, Shannon Zejiang Shen, Hamish Ivison, Varsha Kishore, Jingming Zhuo, Xinran Zhao, Molly Park, Samuel G Finlayson, David Sontag, et al. Dr tulu: Reinforcement learning with evolving rubrics for deep research. *arXiv preprint arXiv:2511.19399*, 2025.

Manasi Sharma, Chen Bo Calvin Zhang, Chaithanya Bandi, Clinton Wang, Ankit Aich, Huy Nghiem, Tahseen Rabbani, Ye Htet, Brian Jang, Sumana Basu, et al. Researchrubrics: A benchmark of prompts and rubrics for evaluating deep research agents. *arXiv preprint arXiv:2511.07685*, 2025.

Haozhan Shen, Peng Liu, Jingcheng Li, Chunxin Fang, Yibo Ma, Jiajia Liao, Qiaoli Shen, Zilun Zhang, Kangjia Zhao, Qianqian Zhang, et al. Vlm-r1: A stable and generalizable r1-style large vision-language model. *arXiv preprint arXiv:2504.07615*, 2025.

Leheng Sheng, Wenchang Ma, Ruixin Hong, Xiang Wang, An Zhang, and Tat-Seng Chua. Reinforcing chain-of-thought reasoning with self-evolving rubrics. *arXiv preprint arXiv:2602.10885*, 2026.

Gaurav Shinde, Anuradha Ravi, Emon Dey, Shadman Sakib, Milind Rampure, and Nirmalya Roy. A survey on efficient vision-language models. *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery*, 15(3):e70036, 2025.

Aaditya Singh, Adam Fry, Adam Perelman, Adam Tart, Adi Ganesh, Ahmed El-Kishky, Aidan McLaughlin, Aiden Low, AJ Ostrow, Akhila Ananthram, et al. Openai gpt-5 system card. *arXiv preprint arXiv:2601.03267*, 2025.

Clemencia Siro, Pourya Aliannejadi, and Mohammad Aliannejadi. Learning to judge: Llms designing and applying evaluation rubrics. In *Findings of the Association for Computational Linguistics: EACL 2026*, pp. 6371–6389, 2026.

Stone Tao, Fanbo Xiang, Arth Shukla, Yuzhe Qin, Xander Hinrichsen, Xiaodi Yuan, Chen Bao, Xinsong Lin, Yulin Liu, Tse-kai Chan, et al. Maniskill3: Gpu parallelized robotics simulation and rendering for generalizable embodied ai. *arXiv preprint arXiv:2410.00425*, 2024.

Mark Towers, Ariel Kwiatkowski, Jordan Terry, John U Balis, Gianluca De Cola, Tristan Deleu, Manuel Goulão, Andreas Kallinteris, Markus Krimmel, Arjun KG, et al. Gymnasium: A standard interface for reinforcement learning environments. *arXiv preprint arXiv:2407.17032*, 2024.

Weijie Tu, Weijian Deng, and Tom Gedeon. A closer look at the robustness of contrastive language-image pre-training (clip). *Advances in Neural Information Processing Systems*, 36:13678–13691, 2023.

Kangrui Wang, Pingyue Zhang, Zihan Wang, Yaning Gao, Linjie Li, Qineng Wang, Hanyang Chen, Chi Wan, Yiping Lu, Zhengyuan Yang, et al. Vagen: Reinforcing world model reasoning for multi-turn vlm agents. *arXiv preprint arXiv:2510.16907*, 2025a.

Yuhang Wang, Yuling Shi, Mo Yang, Rongrui Zhang, Shilin He, Heng Lian, Yuting Chen, Siyu Ye, Kai Cai, and Xiaodong Gu. Swe-pruner: Self-adaptive context pruning for coding agents. *arXiv preprint arXiv:2601.16746*, 2026.

Zi Wang, Xingqiao Wang, Sangah Lee, and Xiaowei Xu. Arise: Agentic rubric-guided iterative survey engine for automated scholarly paper generation. *arXiv preprint arXiv:2511.17689*, 2025b.

Jialong Wu, Shaofeng Yin, Ningya Feng, and Mingsheng Long. Rlvr-world: Training world models with reinforcement learning. *arXiv preprint arXiv:2505.13934*, 2025.

Peng Xia, Kaide Zeng, Jiaqi Liu, Can Qin, Fang Wu, Yiyang Zhou, Caiming Xiong, and Huaxiu Yao. Agent0: Unleashing self-evolving agents from zero data via tool-integrated reasoning. *arXiv preprint arXiv:2511.16043*, 2025.

Zhishang Xiang, Chengyi Yang, Zerui Chen, Zhimin Wei, Yunbo Tang, Zongpei Teng, Zexi Peng, Zongxia Li, Chengsong Huang, Yicheng He, et al. A systematic survey of self-evolving agents: From model-centric to environment-driven co-evolution. 2026.

Ran Xu, Tianci Liu, Zihan Dong, Tony Yu, Ilgee Hong, Carl Yang, Linjun Zhang, Tao Zhao, and Haoyu Wang. Alternating reinforcement learning for rubric-based reward modeling in non-verifiable llm post-training. *arXiv preprint arXiv:2602.01511*, 2026.

Rui Yang, Hanyang Chen, Junyu Zhang, Mark Zhao, Cheng Qian, Kangrui Wang, Qineng Wang, Teja Venkat Koripella, Marziyeh Movahedi, Manling Li, et al. Embodiedbench: Comprehensive benchmarking multi-modal large language models for vision-driven embodied agents. *arXiv preprint arXiv:2502.09560*, 2025.

Chenlu Ye, Zhou Yu, Ziji Zhang, Hao Chen, Narayanan Sadagopan, Jing Huang, Tong Zhang, and Anurag Beniwal. Beyond correctness: Harmonizing process and outcome rewards through rl training. *arXiv preprint arXiv:2509.03403*, 2025a.

Tianzhu Ye, Li Dong, Xun Wu, Shaohan Huang, and Furu Wei. On-policy context distillation for language models. *arXiv preprint arXiv:2602.12275*, 2026.

Zhiling Ye, Yun Yue, Haowen Wang, Xudong Han, Jiadi Jiang, Cheng Wei, Lei Fan, Jiaxin Liang, Shuowen Zhang, Ji Li, et al. Self-rewarding rubric-based reinforcement learning for open-ended reasoning. *arXiv preprint arXiv:2509.25534*, 2025b.

Yunpeng Zhai, Shuchang Tao, Cheng Chen, Anni Zou, Ziqian Chen, Qingxu Fu, Shinji Mai, Li Yu, Jiaji Deng, Zouying Cao, et al. Agentevolver: Towards efficient self-evolving agent system. *arXiv preprint arXiv:2511.10395*, 2025.

Qiyuan Zhang, Junyi Zhou, Yufei Wang, Fuyuan Lyu, Yidong Ming, Can Xu, Qingfeng Sun, Kai Zheng, Peng Kang, Xue Liu, et al. Rubricbench: Aligning model-generated rubrics with human standards. *arXiv preprint arXiv:2603.01562*, 2026a.

Shengtao Zhang, Jiaqian Wang, Ruiwen Zhou, Junwei Liao, Yuchen Feng, Zhuo Li, Yujie Zheng, Weinan Zhang, Ying Wen, Zhiyu Li, et al. Memrl: Self-evolving agents via runtime reinforcement learning on episodic memory. *arXiv preprint arXiv:2601.03192*, 2026b.

Shuo Zhang, Chaofa Yuan, Ryan Guo, Xiaomin Yu, Rui Xu, Zhangquan Chen, Zinuo Li, Zhi Yang, Shuhao Guan, Zhenheng Tang, et al. Evofsm: Controllable self-evolution for deep research with finite state machines. *arXiv preprint arXiv:2601.09465*, 2026c.

Wenlin Zhang, Xiangyang Li, Kuicai Dong, Yichao Wang, Pengyue Jia, Xiaopeng Li, Yingyi Zhang, Derong Xu, Zhaocheng Du, Huifeng Guo, et al. Process vs. outcome reward: Which is better for agentic rag reinforcement learning. *arXiv preprint arXiv:2505.14069*, 2025.

Yao Zhang, Shijie Tang, Zeyu Li, Zhen Han, and Volker Tresp. Webarbiter: A principle-guided reasoning process reward model for web agents. *arXiv preprint arXiv:2601.21872*, 2026d.

Fangqi Zhu, Zhengyang Yan, Zicong Hong, Quanxin Shou, Xiao Ma, and Song Guo. Wmpo: World model-based policy optimization for vision-language-action models. *arXiv preprint arXiv:2511.09515*, 2025.

A Training Procedure and Reward Validatio

A.1 Algorithm Pipeline

Algorithm 1 GEAR: Co-evolving Policies and Grounded Rubrics

Input: Task set $\mathcal{D}$, Environment $\mathcal{E}$, Expert data $\mathcal{D}_{\text{exp}}$
Stage 1: Expert-alignment SFT (Cold Start)
Initialize π_θ^{Sol} and π_θ^{Rub} via SFT on $\mathcal{D}_{\text{exp}}$ to establish semantic foundation
Stage 2: Co-evolving RL Training
for each RL iteration **do**
    Interaction: Collect trajectories $\tau \sim \pi_\theta^{\text{Sol}}(\cdot \mid x, h_t)$ from $\mathcal{E}$
    Generation: Rubricator π_θ^{Rub} proposes candidate rubrics $\hat{\rho}_k \in \widehat{\mathcal{R}}$
    Verification: For each fixed $\hat{\rho}_k$, compute $\mathbf{v}_k$ and $\mathbf{z}$ over its comparison group (Appendix A.2)
    Validation: Filter $\mathcal{R}^{\text{valid}} \leftarrow \{\hat{\rho}_k \mid \text{corr}(\mathbf{v}_k, \mathbf{z}) > \alpha \ \wedge \ \text{std}(\mathbf{v}_k) > 0 \ \wedge \ \text{std}(\mathbf{z}) > 0\}$
    Scheduling: Determine λ using warmup and historical alignment (Appendix A.3)
    Reward Calculation:
    $r^{\text{sol}} \leftarrow r^{\text{env}} + \lambda r^{\text{crit}}$ (Solver: Environment + Validated Rubrics)
    $r^{\text{rub}} \leftarrow r^{\text{rub}}_{\text{align}} + \eta_{\text{format}} r^{\text{rub}}_{\text{format}}$ (Rubricator: Alignment + Format)
    Optimization: Update shared backbone θ via joint PPO objective:
    $J(\theta) = J_{\text{sol}}(\theta) + \omega J_{\text{rub}}(\theta)$
end for
return optimized shared VLM policy π_θ

A.2 Correlation Estimation and Reward Selection

For each sampled task instance, the rollout code groups trajectories with the same prompt. In the reported n-way training configuration, the nominal group contains eight trajectories. The policy rubricator may propose a different set of criteria for each source trajectory; each proposed criterion is then judged on every trajectory in its group. For criterion ρ_k, let $v_{k,j} \in \{0, 1\}$ be the verifier judgment and z_j the environment-derived outcome for group member j. GEAR computes Pearson

correlation between $(v_{k,j})_j$ and $(z_j)_j$ when both vectors vary, and accepts the criterion if the estimate is greater than α. Sokoban and FrozenLake training scripts set trajectory success as the outcome target; the SVG script sets environment reward. A constant satisfaction vector is rejected when group outcomes vary. If all trajectories in a comparison group have the same environment outcome, their rubric satisfaction cannot be correlated with success within that group. For example, eight unsuccessful Sokoban trajectories provide no variation in the success target. When historical filtering is enabled, GEAR uses a sufficiently supported correlation previously estimated for the same task and rule; if no such estimate is available, the candidate is not selected for auxiliary reward in that batch. The historical tracker is keyed by task, canonical rule ID, and correlation-schema version; it stores exponentially decayed satisfaction–outcome moments and is updated after all candidates in a batch have been scored.

A.3 Reward Aggregation and Scheduling

The implemented verifier judges a criterion against the current visual context and the solver response, using a binary JSON judgment. It does not receive the outcome target z as a verifier label in the online request. Generated rubric weights are clamped to $[-20, 20]$; positive weights are then normalized to sum to one, while negative weights remain penalties. The selected rubric score is the weighted satisfaction sum scaled to $[0, 1]$ by its attainable minimum and maximum. The reward function combines this sample-level score with the environment outcome. The trainer places each resulting scalar at the final response token of the corresponding PPO sample, rather than distributing it across every token. Warmup linearly increases the configured rubric-reward weight in runs that enable it. Correlation gating, plateau detection, and recovery are optional switches, so their thresholds should not be presented as universal settings for every reported run.

B Cold-start (SFT) Configuration

B.1 Expert-alignment SFT

To equip the initial model with fundamental reasoning and evaluative capabilities, we conduct a cold-start supervised fine-tuning (SFT) phase. The training corpus consists of 40,000 high-quality trajectories synthesized from the Doubao-Seed-1.6-thinking engine. Specifically, this dataset is bifurcated into 20,000 instances of mathematical reasoning and 20,000 instances of rubric-based evaluation traces. All samples are curated via reject sampling to ensure the logical integrity and correctness of the expert demonstrations.

Table 4: Configuration of key training hyperparameters for the cold-start SFT stage.

Hyperparameter	Value	Technical Specification
Model Backbone	Qwen2.5-VL-3B	Multimodal vision-language base model.
Prompt Template	Qwen2.5-VL	Standardized instruction-following format.
Max Sequence Length	32,768	Maximum context window for long-horizon traces.
Batch Size (per device)	1	Micro-batch size optimized for memory efficiency.
Grad. Accumulation	1	Step-wise gradient updates for stable convergence.
Learning Rate	2e-5	Cosine decay schedule with a minimum of 2e-6.
Weight Decay	0.1	Regularization factor to prevent overfitting.
Warmup Ratio	0.01	Initial learning rate ramp-up period.
Training Epochs	5.0	Total passes over the expert dataset.

We perform full-parameter SFT on the Qwen2.5-VL-3B backbone using this specialized dataset. The training is conducted with a sequence cutoff length of 32,768 tokens to accommodate long-chain reasoning. The comprehensive hyperparameter configurations for this stage are detailed in Table 4.

B.2 Verifier Distillation and Evaluation

The verifier is distilled from 4,000 Gemini 2.5 Pro-labeled rubric–trajectory pairs, covering Spatial Grounding, Action Legality, Temporal Consistency, and Transition Progress with uniform distribution. The student model is Qwen2.5-VL-3B, and its prediction indicates whether a trajectory satisfies a

given rubric. These pairs provide supervision for verifier distillation, separately from the 40,000 cold-start examples described above.

To assess verifier reliability, we evaluate Gemini 2.5 Pro and the distilled verifier on a disjoint human-annotated set of 467 examples. We also replace the distilled verifier with Qwen3-VL-32B-Instruct while keeping the remaining pipeline unchanged. Table 5 summarizes judgment accuracy and downstream task performance.

Table 5: Verifier evaluation and downstream task performance. Sokoban and FrozenLake scores are success rates. Qwen2.5-VL-3B-Distilled is trained using Gemini 2.5 Pro labels. A dash indicates an unreported result.

Verifier	Accuracy (%)	Macro-F1 (%)	Sokoban	FrozenLake
Gemini 2.5 Pro	98.29	97.91	0.88	0.84
Qwen2.5-VL-3B-Distilled	96.36	95.45	0.86	0.83
Qwen3-VL-32B-Instruct	48.82	43.01	0.82	0.78

The distilled verifier achieves 96.36% accuracy, 1.93 percentage points below Gemini. Replacing it with Qwen3-VL-32B-Instruct reduces task success by 4–5 percentage points, showing that GEAR retains useful performance with the lower-accuracy verifier. This comparison does not isolate the contribution of filtering from the retained environment reward. Offline teacher supervision is still used, while online RL and deployment require no proprietary API calls.

C Additional Evaluation Results

C.1 Three-Seed Aggregate Results

We report mean $\pm$ standard deviation over training seeds 42, 43, and 44 on Sokoban, FrozenLake, SVG, and Navigation. This four-task analysis is separate from the five-task point estimate in Table **??**, which also includes PrimitiveSkill.

Table 6: Four-task aggregate results over three training seeds.

Method	Sokoban	FrozenLake	SVG	Navigation	Four-task mean
PPO	0.182 ± 0.016	0.204 ± 0.027	0.634 ± 0.011	0.289 ± 0.022	0.327 ± 0.019
SFT	0.317 ± 0.008	0.218 ± 0.015	0.668 ± 0.006	0.411 ± 0.016	0.404 ± 0.011
Static Rubrics	0.779 ± 0.012	0.781 ± 0.014	0.814 ± 0.005	0.765 ± 0.013	0.785 ± 0.011
VAGEN-Full	0.789 ± 0.018	0.723 ± 0.010	0.794 ± 0.010	0.804 ± 0.013	0.778 ± 0.013
GEAR	0.862 ± 0.010	0.821 ± 0.011	0.826 ± 0.003	0.864 ± 0.005	0.843 ± 0.007

C.2 Held-Out and Size-Shift Results

For procedural grid tasks, training seeds are 1–10,000, validation seeds 10,001–10,256, and fixed test seeds beginning at 20,001. The test seeds are excluded from policy training, verifier distillation, hyperparameter selection, and early stopping. We evaluate 256 episodes per setting without additional GEAR fine-tuning. Sokoban compares unseen 6×6 one-box layouts with 7×7 size-shift layouts, both requiring 1–5 solution steps. FrozenLake compares unseen 4×4 layouts with 5×5 size-shift layouts at matched 1–6-step difficulty.

Table 7: Success rates on held-out and size-shift layouts.

Method	Sokoban held-out	Sokoban size-shift	FrozenLake held-out	FrozenLake size-shift
VAGEN-Full	0.67	0.45	0.52	0.48
GEAR	0.83	0.61	0.72	0.67

C.3 Representative Rubric Decisions and Validation Dynamics

Table 8 gives logged rubric examples. The correlation values describe these instantiations on their sampled comparison groups. Table 9 reports validation at four checkpoints. Accepted and rejected alignment are mean Pearson correlations among criteria with computable correlations. These examples and dynamics complement the w/o Validation ablation: the unfiltered variant also has rubric-based reward shaping, whereas full GEAR selects criteria using environment feedback.

Table 8: Accepted and rejected rubric examples.

Task	Candidate criterion	Decision	Correlation
Sokoban	Legally move a box closer to an unfilled target without causing an irreversible deadlock or blocking future pushes.	Accepted	1.00
Sokoban	Initial turn; no history to reference; auto-PASS.	Rejected	−0.65
FrozenLake	Detect that moving Down reaches a hole and choose a safe alternative, such as Right.	Accepted	0.38
FrozenLake	No holes detected; auto-PASS.	Rejected	−1.00

Table 9: Rubric validation dynamics during training.

Step	Accepted rate	Accepted alignment	Rejected alignment	Solver success
1	0.351	0.431	−0.042	0.35
100	0.753	0.686	−0.218	0.75
200	0.816	0.789	−0.548	0.83
250	0.830	0.717	−0.425	0.86

C.4 Shared-Backbone Cost Comparison

We compare separate and shared optimization as an empirical efficiency–performance trade-off. The shared setting achieves 0.86 Sokoban success versus 0.87 for separate optimization, with reported peak memory 431.5 versus 783.2 GB and compute 29.5 versus 31.2 GPU hours. The deployed parameter counts are 3B and 6B, respectively. GPU hours denote aggregate compute consumption rather than wall-clock duration; the concurrent rise in solver success and rubric acceptance does not by itself prove an absence of gradient conflict.

C.5 Zero-Shot Reference Comparison

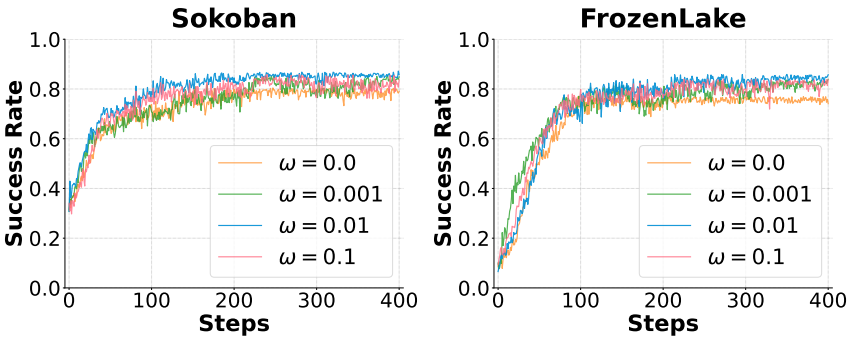
The proprietary references were given the same system and task prompts, visual observations, interaction histories, action and turn limits, test instances, and decoding settings used in the comparison. Nevertheless, the references are zero-shot whereas GEAR is trained on the tasks, so their scores are task-level reference points rather than evidence of general model superiority.

D Additional Hyperparameter Analysis

D.1 Correlation Threshold Sensitivity

For each rubric ρ_k, we evaluate the same criterion across the N trajectories used to construct $\mathbf{v}_k$ and $\mathbf{z}$. Correlation is computed only when both rubric satisfaction and environment outcomes have nonzero variance. Rubric validity is determined at the individual-rubric level, whereas the dimension-level tracker monitors historical alignment and controls the strength of rubric supervision. The threshold α serves as an empirical selection criterion rather than a statistical significance level. When selection and reward computation use the same sampled group, the measured correlation does not constitute independent evidence of validity on unseen trajectories.

Table 10 examines sensitivity to α on Sokoban, FrozenLake, SVG, and Navigation. Performance varies non-monotonically, with $\alpha = 0.1$ attaining the highest reported score on each task and $\alpha = 0.2$



(a) Sokoban (b) FrozenLake

Figure 4: Sensitivity analysis of ω.

yielding the same score on SVG. These results suggest an empirical trade-off between the amount and selectivity of auxiliary supervision; they do not establish reward hacking as the cause of performance differences.

Table 10: Sensitivity to the correlation threshold α.

α	Sokoban	FrozenLake	SVG	Navigation
0.0	0.82	0.79	0.80	0.79
0.01	0.84	0.80	0.82	0.85
0.1	0.86	0.82	0.83	0.86
0.2	0.85	0.81	0.83	0.84
0.5	0.83	0.80	0.81	0.81

D.2 Rubricator Optimization Strength.

We analyze GEAR's sensitivity to the rubricator optimization strength (ω), which balances criteria adaptation with policy evolution. As shown in Figure 4, performance remains robust across various scales, peaking at $\omega = 0.01$. While $\omega = 0$ leads to stagnation in long-horizon tasks, excessive values can trigger instability as the rubricator may overfit to exploration noise. These results underscore a key advantage of GEAR: its inherent stability in aligning self-generated rewards with environment dynamics without intensive hyper-parameter tuning. This controlled optimization ensures discriminative supervision while maintaining stable policy gradients.

E Evaluation Metrics

To rigorously assess agent performance across diverse tasks, we implement a multi-dimensional evaluation suite. Metrics are categorized based on the nature of the task environment into *Task-Completion* and *Visual Fidelity* metrics.

E.1 Metrics for Task-Completion Environments

For logic-based puzzles (e.g., Sokoban, FrozenLake) and embodied control tasks (e.g., Navigation, PrimitiveSkill), the primary metric is the **Success Rate**.

- **Binary Success Indicator** $f(\tau)$: For a given trajectory τ, $f(\tau) = 1$ if the agent successfully achieves the predefined goal, and 0 otherwise. In these environments, rewards are typically sparse and triggered only upon goal completion.
- **Success Rate Formulation**: The success rate is defined as the expected value of the success indicator over a test dataset $\mathcal{D}$:

$$\text{Success Rate} = \mathbb{E}_{\tau \sim \mathcal{D}}[f(\tau)] \tag{25}$$

E.2 Metrics for SVG Reconstruction

In SVG generation tasks, where success is defined by visual adherence rather than discrete goals, we focus on **Perceptual Fidelity**. We utilize two complementary metrics to capture different aspects of visual similarity:

- **DreamSim Score** Fu et al. (2023): Measures the perceptual similarity between the generated and target images using a pre-trained diffusion model baseline $g(\tau) \in [0, 1]$. The score is reported as $\mathbb{E}_{\tau\sim\mathcal{D}}[g(\tau)]$.
- **DINO Score** Caron et al. (2021): Evaluates semantic consistency in a self-supervised visual feature space $h(\tau) \in [0, 1]$. The score is reported as $\mathbb{E}_{\tau\sim\mathcal{D}}[h(\tau)]$.
- **Overall Average**: To provide a holistic assessment, we report the arithmetic mean: $\text{Avg} = (\text{DreamSim} + \text{DINO})/2$.

E.3 Metric Definitions Summary

The key functions defined over a trajectory τ are summarized in Table 11.

Table 11: Summary of trajectory-based evaluation functions.

Function	Range	Description
$f(\tau)$	$\{0, 1\}$	Binary indicator for task completion.
$g(\tau)$	$[0, 1]$	Perceptual similarity via DreamSim.
$h(\tau)$	$[0, 1]$	Semantic consistency via DINO features.

This evaluation protocol follows the standards established by VAGEN Wang et al. (2025a), ensuring comparability and methodological rigor in the assessment of multi-turn VLM agents.

F Details of Environments and Tasks

Cognitive Grid Puzzles: We employ Sokoban Schrader (2018) and FrozenLake Towers et al. (2024) to evaluate logical planning in 2D discrete spaces. In Sokoban, agents must execute precise sequences to maneuver boxes to targets while avoiding deadlocks. In FrozenLake, we disable the "slippery" setting to ensure environment determinism during goal-oriented navigation. Both tasks demand robust state estimation and multi-step reasoning under limited visibility.

Embodied 3D Control: To test high-dimensional visual reasoning, we utilize Navigation Kolve et al. (2017); Yang et al. (2025) and PrimitiveSkill Nasiriany et al. (2022); Tao et al. (2024). Navigation requires locating indoor objects via first-person discrete actions, challenging spatial memory. PrimitiveSkill involves third-person 3D grounding, where the agent maps coordinate-based reasoning (e.g., Panda arm pick-and-place) to physical interactions.

Generative Reasoning: We evaluate open-ended creativity via SVG Reconstruction Rodriguez et al. (2025). The agent is tasked with generating SVG code to replicate a target geometry within an unbounded text-based action space. This environment serves as a stress test for cross-modal consistency, requiring the alignment of abstract linguistic commands with dense, pixel-level visual outcomes.

Table 12: Action space definition for the Sokoban puzzle environment.

Action	Operational Description
Up	Adjust the agent's spatial coordinates one cell upward within the grid.
Left	Translate the agent's position to the immediate left-adjacent cell.
Right	Shift the agent's location by one grid unit to the right.
Down	Relocate the agent to the cell directly below its current position.

Table 13: Hyperparameter configurations and environmental constraints for Sokoban.

Parameter	Value	Description
dim_room	(6, 6)	The spatial configuration of the navigation grid, defining the horizontal and vertical extent of the puzzle area.
max_steps	100	The absolute ceiling for sequential operations permitted within a single experimental episode.
num_boxes	1	The cumulative count of movable entities required to be successfully transitioned to designated target locations.
min_actions_to_succeed	5	The theoretical lower bound of atomic maneuvers necessary to achieve a successful task state from the initial configuration.
max_actions_per_step	3	The operational bandwidth or action capacity available to the agent during each discrete decision-making interval.
max_turns	3	The maximum temporal horizon, expressed in interactive turns, allotted for the agent to engage with the environment.

Table 14: Definition of the action space in FrozenLake.

Action	Description
Up	Shift the agent's position one cell northward.
Left	Displace the agent one cell to the left.
Right	Displace the agent one cell to the right.
Down	Shift the agent's position one cell southward.

Table 15: Environmental parameters for FrozenLake.

Parameter	Value	Description
desc	None	Map topology (randomly generated if unspecified).
is_slippery	False	Boolean flag for stochastic ice surface (slippery) conditions.
size	4	Edge length of the square grid world.
max_actions_per_step	3	Turn-level budget for agent action execution.
min_actions_to_succeed	5	Minimal step count theoretically needed to reach the goal.
max_turns	3	Maximum permitted sequence of interactive turns.

Table 16: Interactive action set for the Navigation environment.

Action	Description
MoveAhead / Back	Translational movement along the agent's longitudinal axis.
MoveRight / Left	Lateral displacement within the environment.
RotateRight / Left	Yaw adjustment in increments of 90 degrees.
LookUp / Down	Pitch adjustment of the camera view by 30-degree steps.

Table 17: System parameters for the Navigation benchmark.

Parameter	Value	Description
resolution	255	Pixel density of the rendered visual input.
down_sample_ratio	1.0	Scaling coefficient for image pre-processing.
fov	100	Angular extent of the agent's observable visual field.
multiview	False	Toggle for concurrent multi-perspective camera streams.
max_actions_per_step	5	Ceiling of actions per decision-making interval.
success_threshold	1.5	Proximity criterion for defining goal achievement.
step_length	0.5	Linear distance traversed in a single movement action.
max_turns	4	Operational turn limit for environmental interaction.

Table 18: Operational definitions of robotic primitives in the PrimitiveSkill environment.

Primitive Command	Robotic Implementation Logic
pick(x, y, z)	Engage the robotic end-effector to capture the target entity at the specified cartesian coordinates (x, y, z) within the operational workspace.
place(x, y, z)	Disengage the gripper to deposit the currently manipulated object at the prescribed destination (x, y, z).
push$(x_1, y_1, z_1, x_2, y_2, z_2)$	Apply a precise directional impulse to translate an object from its origin (x_1, y_1, z_1) toward the target position (x_2, y_2, z_2).

Table 19: Operational constraints and resource allocations for the PrimitiveSkill benchmark.

Parameter	Value	Description
max_actions_per_step	2	The maximum allowable count of basic manipulation primitives executable within a single interaction interval.
max_turns	3	The aggregate turn-level resource allocation designated for successfully fulfilling the complex manipulation objective.

Table 20: Generative action protocols and syntax definitions for SVG Reconstruction.

Action Type	Technical Description
SVG Code	Autonomous production of structured XML-based vector commands that define spatial path coordinates and fundamental geometric entities.

Table 21: Dataset configurations and operational limits for SVG generation.

Parameter	Value	Description
dataset_name	starvector/svg-icons-simple	The specific collection of vector icons utilized as the gold-standard reference for the reconstruction task.
max_turns	2	The maximum number of iterative cycles permitted for the progressive optimization of the generated SVG markup.

G PROMPTS

G.1 RUBRIC GENERATION PROMPT FOR SOKOBAN

The prompt specifies four predefined dimensions and task-specific instructions. The distinction between these instructions and the generated rubric content is reflected in the output schema below.

Rubric Generation Prompt for Sokoban

```
# Role
You are a Rubric Designer for Sokoban. Analyze the board image and
generate evaluation rubrics across based on four **Physical Laws**.

**# STEP 1: GROUNDED ANALYSIS**
1. **Spatial Grounding**: Map $P, B, T$ coordinates. Perform
"between_check": Is $P$ on the push-axis for $B \to T$? If yes, define
"PUSH_SIDE_POS".
2. **Action Legality**: Identify **Illegal** (Wall/OOB) and **Fatal**
(Deadlock) directions.
3. **Temporal Consistency**: Compare with history. Calculate
"freshness" ($f$) and detect "loops" (repeated $P_{pos}$ + action).
4. **Transition Progress**: Calculate $\Delta = \text{dist}_{old} -
\text{dist}_{new}$. Check if "detour" or "multi-step plan" is
justified.


**# STEP 2: PHYSICAL LAW EVALUATION**
- **[L1] Spatial Grounding **:
    - Correct relative coords in "<observation>".
    - If "between_check", "<think>" must cite "reposition" or
"PUSH_SIDE_POS".
    - Mention obstacles within 2 cells.

- **[L2] Action Legality**: Score 0 if agent executes Illegal or Fatal
move.
- **[L3] Temporal Consistency**: PASS if $f < 0.6$ and no unmitigated
loops exist.
- **[L4] Transition Progress**: PASS if $\Delta > 0$. If $\Delta \le
0$, PASS only if "<think>" justifies a "detour" or "plan_depth".

**# OUTPUT FORMAT**
Return ONLY JSON. Escape internal braces: "{{ }}".

{{
  "analysis": {{
   "spatial": { "p": [0, 1], "b": [[2, 3]], "between": bool,
"push_pos": [5, 4] },
    "legality": { "illegal": ["Dir"], "fatal": ["Dir"] },
    "temporal": { "freshness": f, "loop": bool },
    "progress": { "delta": "\\Delta", "strategy": "push/detour/plan" }
  }},
  "rubrics": [
    {{ "id": "SpatialGrounding",  "score": N, "description": "Relative
pos + between-logic" }},
    {{ "id": "ActionLegality", "score": N, "description": "VETO if
illegal/fatal" }},
    {{ "id": "TemporalConsistency", "score": N, "description":
"Freshness vs stagnation" }},
    {{ "id": "TransitionProgress", "score": N, "description": "\Delta
> 0 or justified detour" }}
  ]
}}
```

G.2 Rubric Verification Prompt for Sokoban

The verifier receives a candidate rubric and the interaction evidence used to judge satisfaction. Solver-generated reasoning, answers, and predictions are distinct from environment-returned observations and execution feedback; the field names in the template should not be interpreted as establishing that all of their contents are ground truth.

Rubric Verification Prompt for Sokoban

```
# Role
You are the **Grounded Environment Verifier**. Your goal is to assess
whether a specific reasoning trajectory $\tau$ satisfies a given
candidate rubric $\rho$ by cross-referencing the visual state and
environment feedback.

**# INPUT CONTEXT**
1. **[Candidate Rubric]**: The specific criterion to be verified
(e.g., Spatial, Legality, etc.).
2. **[Agent Trajectory]**: The "<think>" process and the "<answer>"
(action) provided by the Solver.
3. **[Environment Feedback]**: The ground-truth execution results from
the environment (e.g., <Observation>, <Reasoning>, <Answer>,
<Prediction>).

**# VERIFICATION TASKS (Chain-of-Thought)**
For the assigned rubric, perform the following logical checks:

1. **Evidence Extraction**: Identify specific phrases in the agent's
"<think>" that claim to follow the rubric (e.g., "The box is to my
left").
2. **Visual Cross-Check**: Compare these claims against the **[Current
Visual State]**. Are the coordinates and relative positions factually
correct?
3. **Logic-Action Mapping**: Does the chosen "<answer>" align with the
reasoning? (e.g., If the agent says "move left to push," does it
actually move left?).
4. **Environment Consistency**: Does the outcome of the action support
the rubric's intent? (e.g., If the rubric requires "no illegal moves,"
did the environment return a "collision" flag?).

**# PHYSICAL LAW ALIGNMENT**
- **Spatial Grounding**: Is the relative distance and direction
reported in the trace numerically accurate?
- **Action Legality**: Does the trace acknowledge forbidden zones? Is
the selected action within $\mathcal{A}_{legal}$?
- **Temporal Consistency**: Does this turn's reasoning show new info,
or is it a verbatim hallucination from $t-1$?
- **Transition Progress**: If the agent claims progress, did the
$\text{dist}(B, T)$ actually decrease in the environment?

**# OUTPUT FORMAT**
Return a satisfactory score only. You must be strictly binary: 1 for
satisfied, 0 for unsatisfied
```